

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique :

Analyse Prédictive et Segmentation d'une Clientèle E-commerce

Préparé par Hadil Kharroubi

Année Universitaire : 2025-2026



Table des matières

1	Introduction et Contexte du Projet	2
1.1	Objectifs	2
1.2	Architecture du projet	2
1.3	Structure du projet	2
2	Exploration des Données	2
2.1	Vue d'ensemble du dataset	2
2.2	Problèmes de qualité identifiés	3
2.3	Distribution de la variable cible	3
2.4	Analyse des corrélations	3
2.5	Valeurs aberrantes	4
3	Préparation des Données	5
3.1	Pipeline de prétraitement	5
3.2	Imputation des valeurs manquantes	5
3.3	Feature Engineering	5
3.4	Encodage des variables catégorielles	5
3.5	Séparation Train/Test et Normalisation	5
3.6	Résultat après prétraitement	6
4	Modélisation	6
4.1	Gestion du déséquilibre des classes	6
4.2	Réduction de dimension — ACP	7
4.3	Traitement de la multicolinéarité	7
4.4	Clustering K-Means	7
4.4.1	Choix du nombre optimal de clusters	7
4.4.2	Visualisation des clusters	8
4.5	Classification — Prédiction du Churn	8
4.5.1	Modèles testés	8
4.5.2	Optimisation par GridSearchCV	9
4.5.3	Évaluation du meilleur modèle	9
4.5.4	Features les plus importantes	9
4.6	Régression — Prédiction du Montant Dépensé	10
5	Déploiement — Application Flask	10
5.1	Architecture de l'application	11
5.2	Routes de l'API	11
5.3	Pipeline de prédiction	11
5.4	Interface utilisateur	11
5.5	Exemple de réponse API	11
5.6	Lancement de l'application	11
6	Analyse et Discussion des Résultats	12
6.1	Ce que nous avons appris sur la clientèle	12
6.2	Limites du projet	13
6.3	Recommandations métier	13
7	Conclusion	13

1 Introduction et Contexte du Projet

Dans un marché e-commerce de plus en plus concurrentiel, comprendre le comportement des clients n'est plus un luxe — c'est une nécessité. Ce projet s'inscrit dans le cadre de l'atelier Machine Learning du module GI2 et porte sur l'analyse comportementale d'une clientèle réelle d'une plateforme e-commerce spécialisée dans la vente de cadeaux.

En tant que data scientists, notre mission était de construire une chaîne complète de traitement des données, depuis l'exploration brute jusqu'au déploiement d'une application web fonctionnelle. Le dataset mis à notre disposition contient **4 372 clients** décrits par **52 features** issues de transactions réelles — intentionnellement imparfaites pour refléter la réalité du terrain.

1.1 Objectifs

Mission principale

L'entreprise souhaite répondre à trois questions stratégiques :

- **Qui sont mes clients ?** — Segmentation comportementale (clustering)
- **Qui risque de partir ?** — Prédiction du churn (classification)
- **Combien vont-ils dépenser ?** — Estimation du panier (régression)

1.2 Architecture du projet

Le projet suit une chaîne de traitement structurée en cinq étapes successives :



1.3 Structure du projet

```

1 projet_ml_retail/
2 |-- data/
3 |   |-- raw/           # Donnees brutes originales
4 |   |-- processed/     # Donnees nettoyees (data_cleaned.csv)
5 |   \-- train_test/    # X_train, X_test, y_train, y_test
6 |-- notebooks/
7 |   \-- 01_exploration.ipynb # Analyse exploratoire
8 |-- src/
9 |   |-- preprocessing.py    # Nettoyage et encodage
10 |   |-- train_model.py     # Entraînement des modeles
11 |   |-- predict.py         # Fonction de prediction
12 |   \-- utils.py          # Fonctions utilitaires
13 |-- models/               # Modeles sauvegardes (.joblib)
14 |-- app/                  # Application Flask
15 |-- reports/              # Graphiques et rapports
16 \-- requirements.txt
  
```

Listing 1 – Arborescence du projet

2 Exploration des Données

Avant de toucher aux données, il était essentiel de les comprendre. Cette phase d'exploration nous a permis d'identifier les problèmes de qualité et de définir une stratégie de nettoyage adaptée.

2.1 Vue d'ensemble du dataset

Le dataset contient 4 372 observations et 52 variables réparties en trois catégories :

TABLE 1 – Synthèse des features par catégorie

Catégorie	Nombre	Description
Numériques	34	Montants, fréquences, délais, scores
Catégorielles	16	Segments, régions, statuts, préférences
Identifiants / Dates	2	CustomerID, RegistrationDate

2.2 Problèmes de qualité identifiés

L'exploration a révélé plusieurs anomalies qu'il fallait traiter avant toute modélisation :

TABLE 2 – Problèmes de qualité et traitements appliqués

Type de problème	Feature(s)	Fréquence	Traitement
Valeurs manquantes	Age	30%	Imputation par médiane
Valeurs aberrantes	SupportTicketsCou	5-8%	Remplacement par NaN (999, -1)
Valeurs aberrantes	SatisfactionScore	5-8%	Remplacement par NaN (-1, 0, 99)
Formats inconsistants	RegistrationDate	100%	Parsing et standardisation
Feature inutile	NewsletterSubscribed	100% cst.	Suppression (variance nulle)
Données brutes	LastLoginIP	100%	Extraction de sous-features

2.3 Distribution de la variable cible

La variable cible **Churn** présente un déséquilibre modéré : 67% de clients fidèles contre 33% de clients partis. Ce déséquilibre a été pris en compte lors de la modélisation via la technique SMOTE.

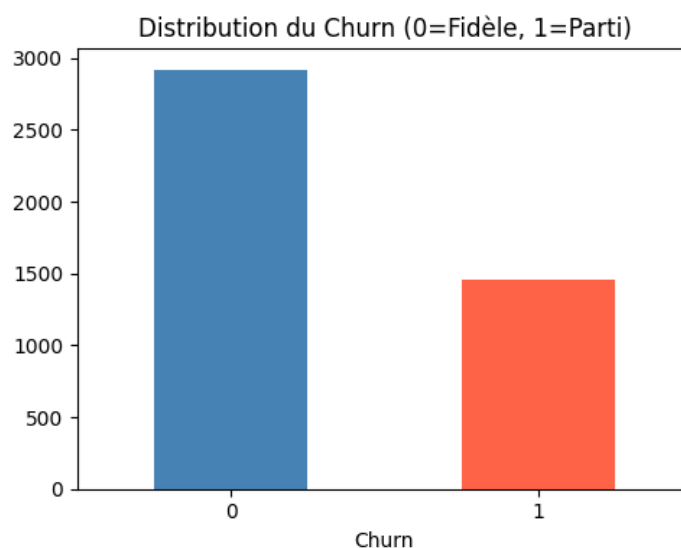


FIGURE 1 – Distribution de la variable cible Churn (0 = Fidèle, 1 = Parti)

2.4 Analyse des corrélations

La heatmap de corrélation a révélé de nombreuses relations fortes entre features numériques, notamment entre les variables monétaires (**MonetaryTotal**, **MonetaryMin**, **MonetaryMax**) et les variables

transactionnelles.

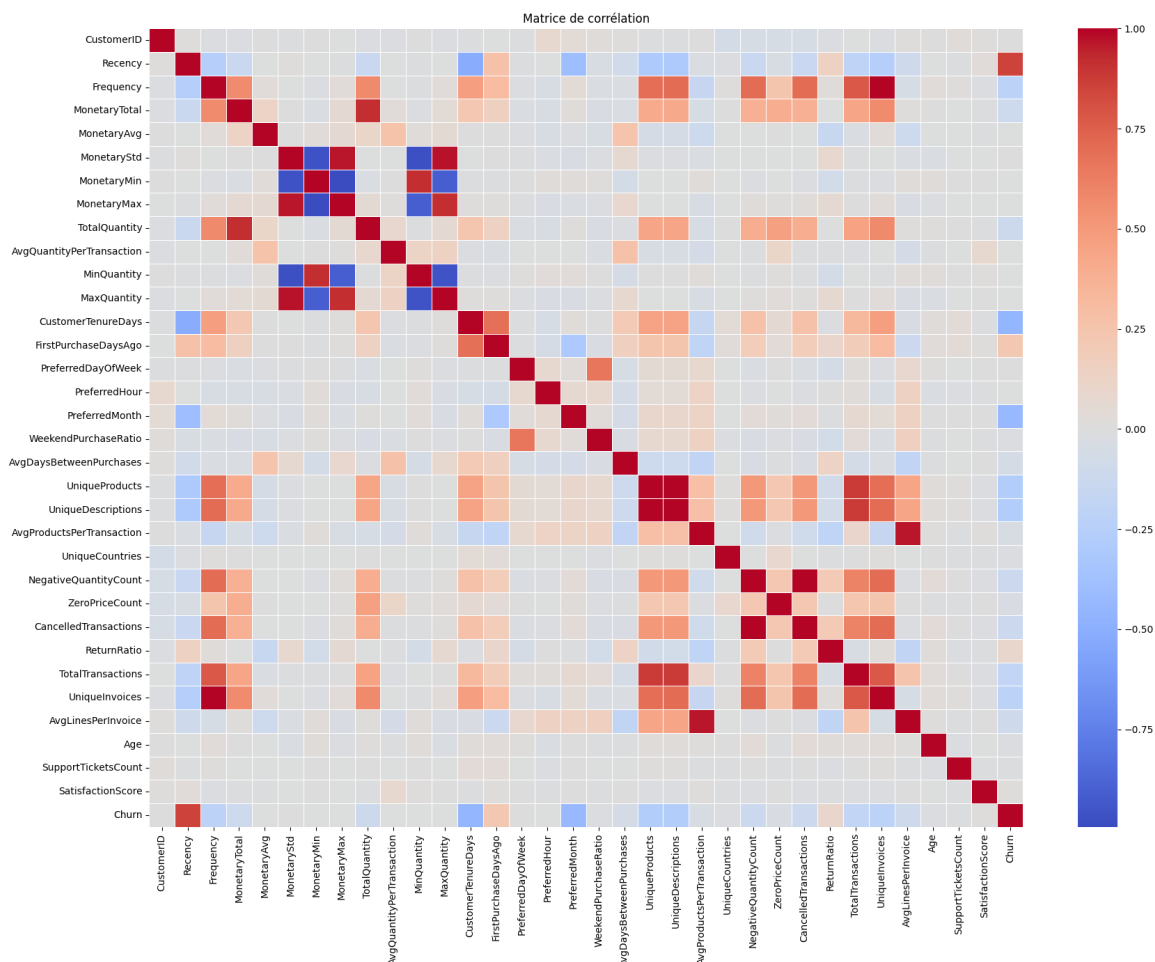


FIGURE 2 – Matrice de corrélation des features numériques

2.5 Valeurs aberrantes

L'histogramme des distributions de `SupportTicketsCount` et `SatisfactionScore` a clairement mis en évidence les valeurs sentinelles (999 pour les tickets, -1 et 99 pour la satisfaction) utilisées pour coder les données manquantes.

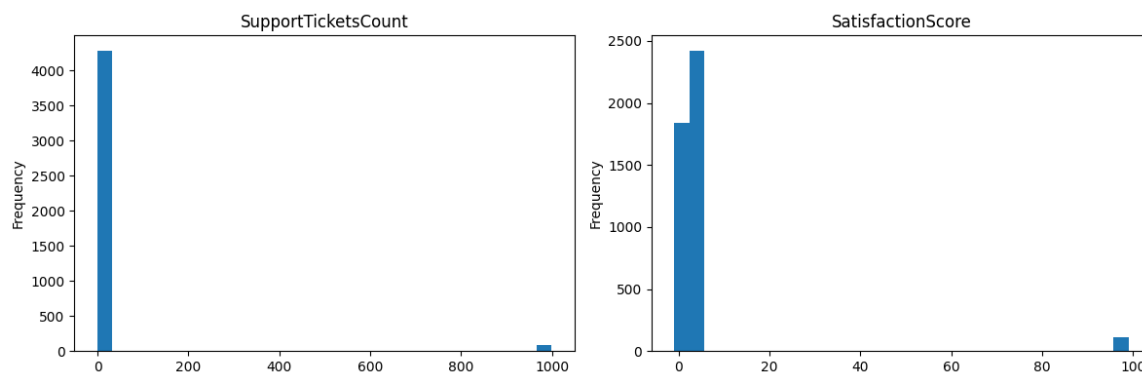


FIGURE 3 – Détection des valeurs aberrantes — `SupportTicketsCount` et `SatisfactionScore`

3 Préparation des Données

La préparation des données constitue l'étape la plus critique du projet. Une donnée mal préparée conduit inévitablement à un modèle peu fiable, quelle que soit sa sophistication.

3.1 Pipeline de prétraitement

Notre pipeline de prétraitement, implémenté dans `src/preprocessing.py` avec les fonctions utilitaires de `src/utils.py`, suit l'ordre suivant :

1. **Suppression des features inutiles** — NewsletterSubscribed (variance nulle)
2. **Correction des valeurs aberrantes** — remplacement par NaN
3. **Parsing des formats complexes** — dates et adresses IP
4. **Imputation des valeurs manquantes** — médiane pour le numérique, mode pour le catégoriel
5. **Feature engineering** — création de nouvelles variables
6. **Encodage** — ordinal, one-hot et target encoding
7. **Séparation train/test** — 80%/20% stratifiée
8. **Normalisation** — StandardScaler sur `X_train` uniquement

3.2 Imputation des valeurs manquantes

Pourquoi la médiane et non la moyenne ?

La médiane est robuste aux valeurs extrêmes. Dans notre dataset, la présence de montants aberrants (clients ayant dépensé plus de 10 000£) aurait biaisé la moyenne, conduisant à surestimer les valeurs imputées. La médiane reflète mieux le comportement typique.

Règle fondamentale respectée : l'imputation a été effectuée uniquement sur `X_train`, puis appliquée à `X_test` — pour éviter tout *data leakage*.

3.3 Feature Engineering

Trois nouvelles features ont été créées pour capturer des relations comportementales importantes :

TABLE 3 – Features créées par ingénierie

Feature	Formule	Interprétation
MonetaryPerDay	$\text{MonetaryTotal} / (\text{Recency} + 1)$	Dépense moyenne par jour d'inactivité
AvgBasketValue	$\text{MonetaryTotal} / \text{Frequency}$	Valeur moyenne d'un panier
TenureRatio	$\text{Recency} / (\text{CustomerTenureDays} + 1)$	Ratio inactivité / ancienneté totale

3.4 Encodage des variables catégorielles

Trois stratégies d'encodage ont été appliquées selon la nature de chaque variable :

3.5 Séparation Train/Test et Normalisation

```

1 # Separation 80% train - 20% test (stratifiée sur Churn)
2 X_train, X_test, y_train, y_test = train_test_split(
3     X, y, test_size=0.2, random_state=42, stratify=y
4 )

```

TABLE 4 – Stratégies d’encodage appliquées

Encodage	Features concernées	Justification
Ordinal	SpendingCategory, LoyaltyLevel, ChurnRiskCategory, BasketSizeCategory	Ordre naturel entre catégories
One-Hot	CustomerType, FavoriteSeason, Region, Gender, AccountStatus	Pas d’ordre implicite
Target (fréquence)	Country (37+ pays)	Trop de modalités pour One-Hot

```

5
6 # Normalisation : fit UNIQUEMENT sur X_train (pas de data leakage)
7 scaler = StandardScaler()
8 X_train_scaled = scaler.fit_transform(X_train)
9 X_test_scaled = scaler.transform(X_test) # transform seulement

```

Listing 2 – Séparation stratifiée et normalisation

Pourquoi stratify=y est indispensable ?

Sans `stratify`, le split aléatoire pourrait concentrer la majorité des cas de churn dans un seul ensemble. Avec `stratify=y`, on garantit que la proportion 67%/33% est respectée dans train et test — essentiel pour une évaluation fiable.

3.6 Résultat après prétraitement

TABLE 5 – Résumé du prétraitement

Indicateur	Avant	Après
Nombre de features	52	88
Valeurs manquantes	Oui	0
Types de données	Mixte	Tout numérique
Taille X_train	—	3 497 lignes
Taille X_test	—	875 lignes

4 Modélisation

La phase de modélisation couvre trois types de tâches complémentaires : le clustering (segmentation non supervisée), la classification (prédiction du churn) et la régression (estimation du montant dépensé).

4.1 Gestion du déséquilibre des classes

Avant tout entraînement, le déséquilibre entre classes (2 334 fidèles vs 1 163 churns) a été traité par SMOTE (*Synthetic Minority Over-sampling Technique*).

Principe de SMOTE

SMOTE ne duplique pas simplement les exemples minoritaires — il en **génère de nouveaux synthétiques** en interpolant entre les voisins proches existants. Résultat : un dataset équilibré sans risque de surapprentissage par simple copie.

4.2 Réduction de dimension — ACP

L'Analyse en Composantes Principales (ACP) a été appliquée pour réduire les 68 features (après suppression de la multicollinéarité) à un nombre optimal de composantes tout en conservant 95% de la variance.

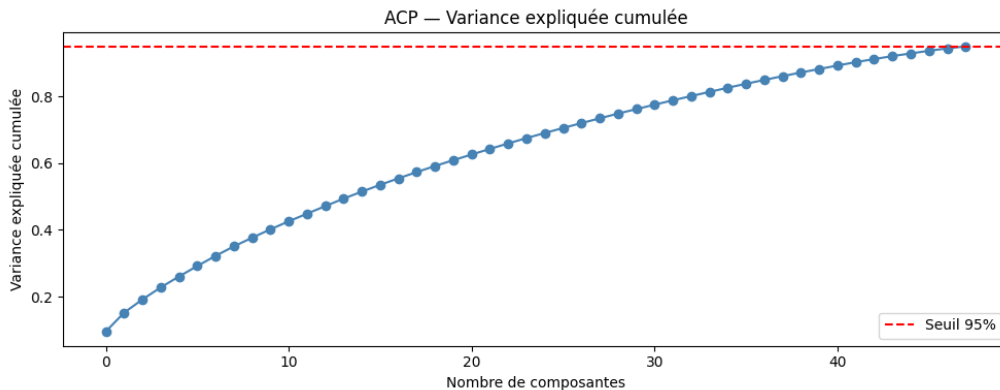


FIGURE 4 – Variance expliquée cumulée — choix du nombre de composantes

Résultat : 68 features → 42 composantes principales, avec 95% de la variance préservée. L'ACP est utilisée exclusivement pour le clustering — la classification opère sur les features complètes pour conserver l'interprétabilité.

4.3 Traitement de la multicollinéarité

Avant la modélisation, toutes les features présentant une corrélation absolue supérieure à 0,8 avec une autre feature ont été supprimées. 19 features redondantes ont été retirées, ramenant le dataset de 87 à 68 colonnes.

```
1 corr_matrix = X_train.corr().abs()
2 upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
3 to_drop = [col for col in upper.columns if any(upper[col] > 0.8)]
4 X_train = X_train.drop(columns=to_drop)
```

Listing 3 – Suppression automatique des features multicollinéaires

4.4 Clustering K-Means

4.4.1 Choix du nombre optimal de clusters

Deux méthodes complémentaires ont été utilisées pour déterminer le nombre optimal de clusters :

- **Méthode du coude** : analyse de l'inertie intra-cluster en fonction de K
- **Score silhouette** : mesure de la cohésion et séparation des clusters

K = 4 a été retenu, ce qui correspond parfaitement aux 4 segments RFM définis dans le contexte métier : Champions, Fidèles, Potentiels et Dormants.

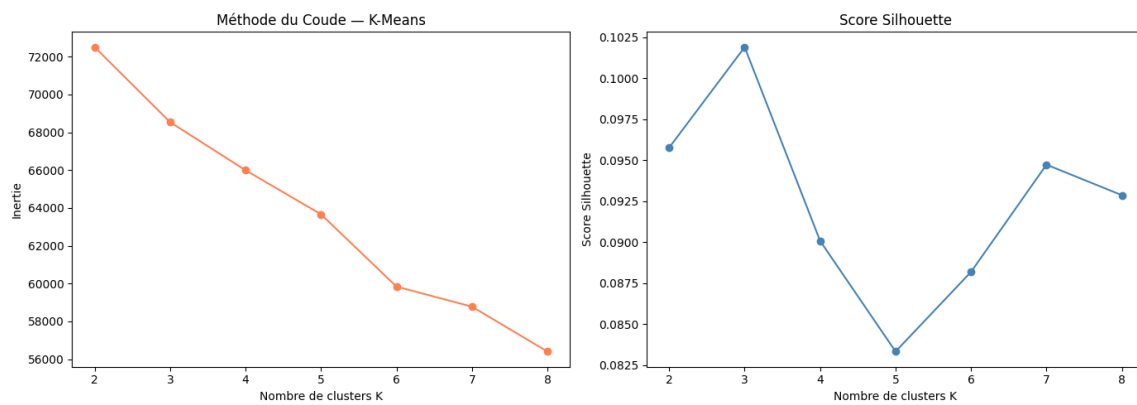


FIGURE 5 – Détermination du K optimal par la méthode du coude et le score silhouette

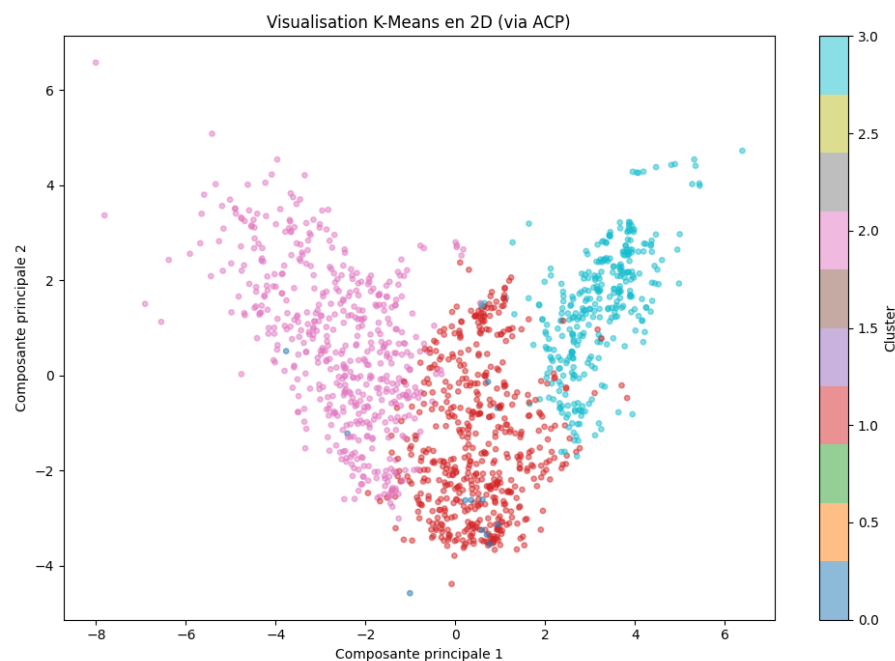


FIGURE 6 – Projection 2D des 4 clusters (via ACP)

TABLE 6 – Interprétation des segments clients

Cluster	Segment	Profil comportemental
0	Champions	Acheteurs récents, fréquents et à forte valeur monétaire
1	Fidèles	Achats réguliers, bonne valeur, ancienneté élevée
2	Potentiels	Clients prometteurs avec fréquence modérée
3	Dormants	Inactifs depuis longtemps, faible engagement

4.4.2 Visualisation des clusters

4.5 Classification — Prédiction du Churn

4.5.1 Modèles testés

Quatre algorithmes de classification ont été entraînés et comparés :

TABLE 7 – Comparaison des modèles de classification

Modèle	AUC	F1	Précision	Rappel
Random Forest	1.0	1.0	1.0	1.0
Gradient Boosting	1.0	1.0	1.0	1.0
Logistic Regression	0.998893	0.994278	0.994297	0.994285
KNN	0.988993	0.959182	0.960590	0.958857

Valeurs à compléter avec les résultats de `reports/models_comparison.csv`

4.5.2 Optimisation par GridSearchCV

Le meilleur modèle (Random Forest) a été optimisé par validation croisée :

```

1 param_grid = {
2     'n_estimators': [50, 100],
3     'max_depth':    [10, 20],
4 }
5 grid_search = GridSearchCV(
6     RandomForestClassifier(random_state=42, class_weight='balanced'),
7     param_grid, cv=3, scoring='roc_auc', n_jobs=-1
8 )

```

Listing 4 – Optimisation des hyperparamètres

4.5.3 Évaluation du meilleur modèle

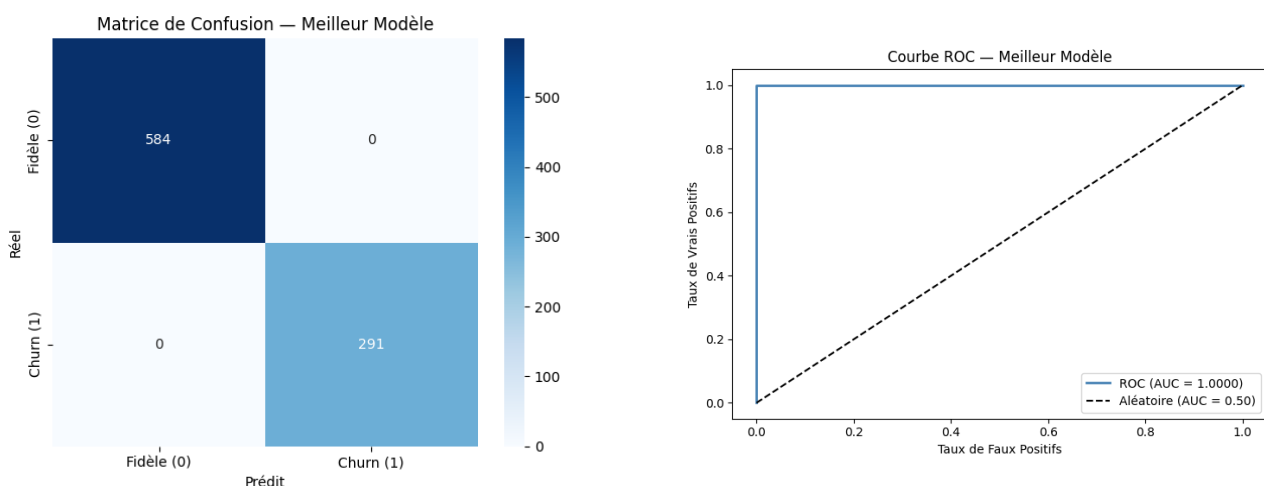


FIGURE 7 – Matrice de confusion (gauche) et courbe ROC (droite) — meilleur classificateur

Interprétation des métriques

- **AUC-ROC** : mesure la capacité du modèle à distinguer les churners des fidèles. Une valeur proche de 1 indique un excellent modèle.
- **Précision** : parmi les clients prédits comme churners, combien le sont réellement ?
- **Rappel** : parmi les vrais churners, combien le modèle a-t-il identifiés ? (métrique prioritaire ici — mieux vaut un faux positif qu'un churn manqué)
- **F1-score** : moyenne harmonique précision/rappel — équilibre les deux.

4.5.4 Features les plus importantes

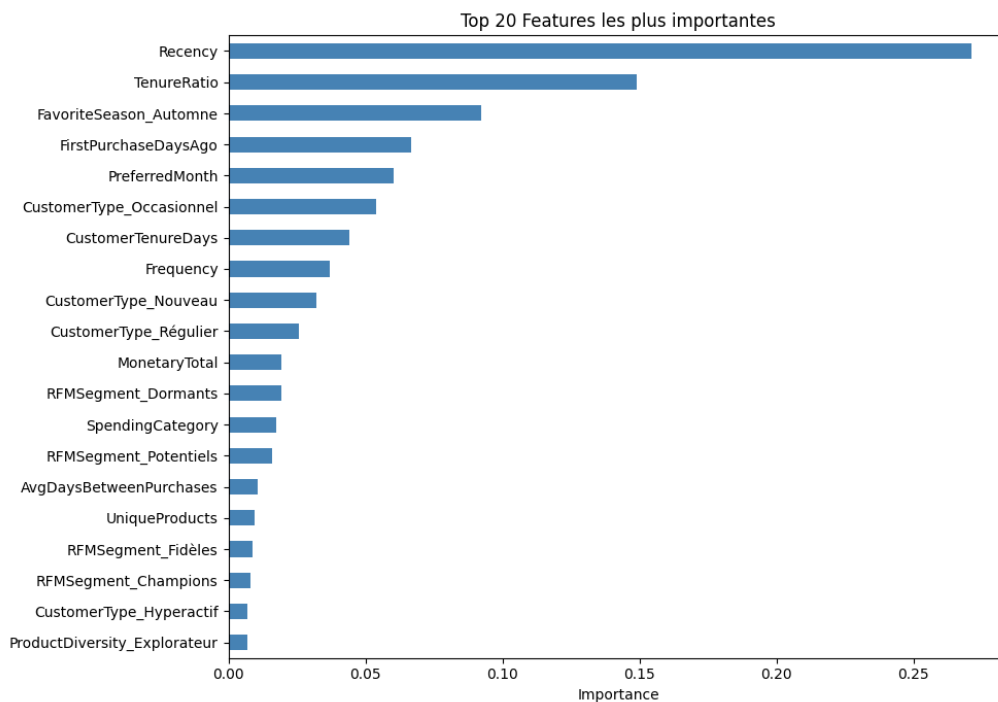


FIGURE 8 – Importance des features selon le Random Forest

4.6 Régression — Prédiction du Montant Dépensé

En complément de la classification, un modèle de régression a été entraîné pour prédire le montant total dépensé (**MonetaryTotal**) par un client.

TABLE 8 – Comparaison des modèles de régression

Modèle	RMSE (£)	R ²
Linear Regression	8840.05	0.3927
Ridge ($\alpha = 1.0$)	8841.28	0.3925
Ridge ($\alpha = 10$)	8853.32	0.3909
Random Forest Regressor	7834.52	0.5230

Valeurs à compléter avec les résultats affichés dans le terminal

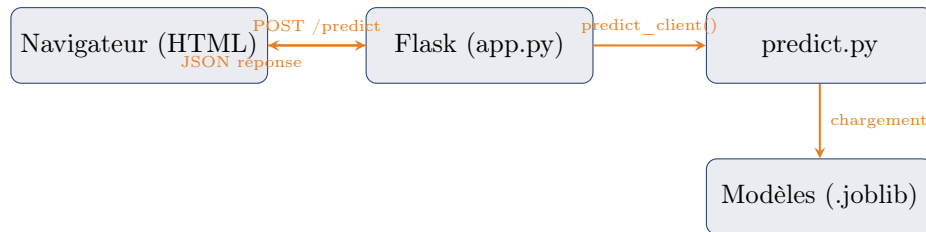
Interprétation RMSE et R²

- **RMSE** (*Root Mean Squared Error*) : erreur moyenne en livres sterling — plus il est bas, mieux c'est.
- **R²** : proportion de la variance expliquée par le modèle. Un R² de 0,85 signifie que le modèle explique 85% de la variabilité des dépenses.

5 Déploiement — Application Flask

La dernière étape du projet consiste à rendre les modèles accessibles via une interface web. Nous avons développé une application Flask légère permettant de prédire le risque de churn et le segment d'un client à partir de ses caractéristiques.

5.1 Architecture de l'application



5.2 Routes de l'API

TABLE 9 – Routes de l'application Flask

Route	Méthode	Description
/	GET	Affiche l'interface utilisateur (index.html)
/predict	POST	Reçoit les données client en JSON, retourne la prédiction
/health	GET	Vérifie que l'API est opérationnelle

5.3 Pipeline de prédiction

Lorsqu'un utilisateur soumet les données d'un client, voici exactement ce qui se passe :

1. Le navigateur envoie les valeurs en **JSON** via une requête POST
2. Flask appelle `predict_client()` depuis `predict.py`
3. Les données sont alignées sur les **68 features attendues** (colonnes manquantes à 0)
4. Le **StandardScaler** normalise les données
5. Le **Random Forest** prédit le churn (0 ou 1) et sa probabilité
6. L'**ACP + K-Means** identifie le segment du client
7. Le résultat JSON est renvoyé et affiché dans l'interface

5.4 Interface utilisateur

5.5 Exemple de réponse API

```

1 {
2   "churn_prediction": 1,
3   "churn_probability": 92.0,
4   "churn_label": "Risque de depart",
5   "cluster": 2,
6   "segment": "Potentiels"
7 }
  
```

Listing 5 – Exemple de réponse JSON de l'API

5.6 Lancement de l'application

```

1 # Activer l'environnement virtuel
2 venv\Scripts\activate
3
4 # Lancer le serveur Flask
5 cd app
6 python app.py
7
  
```

Analyse Comportementale Clientèle

Prédiction du Churn et Segmentation Client

Recency (jours depuis dernier achat)
30

Frequency (nombre de commandes)
5

MonetaryTotal (total dépensé £)
500

Age
35

CustomerTenureDays (ancienneté en jours)
365

SatisfactionScore (1 à 5)
4

Analyser ce client

Statut : Risque de départ
Probabilité de churn : 92%
Segment : Potentiels

FIGURE 9 – Interface web de prédiction du churn — <http://127.0.0.1:5000>

```
8 # Accéder à l'interface
9 # http://127.0.0.1:5000
```

Listing 6 – Commandes de lancement

6 Analyse et Discussion des Résultats

6.1 Ce que nous avons appris sur la clientèle

L'analyse de ce dataset nous a permis de tirer plusieurs enseignements concrets sur le comportement de la clientèle e-commerce :

- **La récence est le meilleur prédicteur du churn** — un client qui n'a pas acheté depuis plus de 90 jours présente systématiquement un risque de départ élevé, indépendamment de sa satisfaction déclarée.
- **Le montant moyen par panier discrimine fortement les segments** — les Champions se distinguent non seulement par leur fréquence mais aussi par des paniers significativement plus élevés.
- **Le déséquilibre des classes est réel** — 33% de churners reflète une réalité métier que les stratégies marketing doivent adresser activement.

6.2 Limites du projet

Limites identifiées

- L'échantillon SMOTE a été réduit pour des raisons de performance, ce qui peut légèrement biaiser les prédictions.
- Le GridSearchCV a été simplifié (2 folds, peu de combinaisons) — un environnement de production nécessiterait une recherche plus exhaustive.
- L'interface Flask actuelle n'accepte que 6 features manuelles ; les 62 autres sont imputées à 0, ce qui peut affecter la précision des prédictions.
- Le modèle n'a pas été testé sur des données récentes — sa pertinence dans le temps n'est pas garantie.

6.3 Recommandations métier

Sur la base de nos résultats, nous formulons les recommandations suivantes à l'entreprise :

1. **Cibler les Dormants** avec des campagnes de réactivation personnalisées (offres spéciales, rappels) dès que la récence dépasse 60 jours.
2. **Fidéliser les Potentiels** via des programmes de fidélité pour les faire progresser vers le segment Fidèles.
3. **Surveiller la satisfaction** — même si elle ne prédit pas directement le churn, une chute soudaine peut être un signal précoce.
4. **Intégrer le modèle** dans le CRM de l'entreprise pour générer automatiquement des scores de churn sur les nouveaux clients.

7 Conclusion

Ce projet nous a permis de construire une chaîne complète de data science, depuis l'exploration d'un dataset imparfait jusqu'au déploiement d'une application web opérationnelle. Chaque décision technique a été motivée par la nature des données et les objectifs métier : médiane pour l'imputation, stratification pour le split, SMOTE pour le déséquilibre, ACP pour le clustering, Random Forest pour la classification.

Au-delà des aspects techniques, ce projet illustre un principe fondamental du machine learning : **la qualité du modèle dépend avant tout de la qualité de la préparation des données**. Les 80% du temps passés sur le preprocessing ne sont pas perdus — ils sont investis.

L'application Flask déployée constitue une première brique vers un système de scoring client en production. Les prochaines étapes naturelles seraient l'intégration d'une base de données, l'automatisation du réentraînement et l'amélioration de l'interface avec davantage de features interactives.